

GUIA PRÁTICO · PASSO A PASSO

Rodando o projeto no **VSCode**

SAD ENCCEJA — Sistema de Apoio à Decisão com o algoritmo K-NN sobre microdados INEP 2024. Este guia conduz a instalação e a execução das duas interfaces do projeto, do ambiente virtual até o navegador.

Interface 1 · Streamlit

Projeto Python completo · porta 8501

Interface 2 · Next.js

Dashboard web · porta 3000

Algoritmo · K-NN

scikit-learn + TypeScript

16,7

MAE global do modelo (escala 0–500)

k = 21

vizinhos escolhidos por validação cruzada

71,5%

acurácia do classificador de aprovação

O que você vai rodar

O projeto entrega um **Sistema de Apoio à Decisão (SAD)** que prevê o desempenho de novos candidatos do ENCCEJA com o algoritmo K-Nearest Neighbors e converte a previsão em recomendações gerenciais para cursinhos preparatórios. Ele é composto por duas interfaces independentes que compartilham a mesma base de dados e a mesma lógica de modelagem. Não é necessário rodar as duas para avaliar o trabalho: cada uma cobre o ciclo completo de previsão e recomendação.

● Interface 1 — App Streamlit (Python)

Carrega o modelo K-NN treinado pelo pipeline (`modelo_knn.joblib`) e oferece três abas: Simulador de Matrícula, Base Histórica e Modelo & Métricas. Roda em `http://localhost:8501`.

● Interface 2 — Dashboard Next.js (Web)

Replica o K-NN inteiramente no navegador em TypeScript, com cinco módulos: Visão Geral, Simulador K-NN (com presets de demonstração), Recomendação, Análise Exploratória e Modelo & Métricas. Roda em `http://localhost:3000`.

O fluxo de dados é simples de entender: o **pipeline Python** lê os microdados (sintéticos ou oficiais do INEP), aplica o ETL, normaliza as variáveis com Z-score, treina o K-NN e serializa o modelo; o **Streamlit** consome esse modelo treinado; já o **dashboard Next.js** recebe uma amostra exportada da base com os parâmetros de padronização e executa a mesma aritmética de vizinhos no cliente. Por isso, o Streamlit depende da etapa de treinamento, enquanto o dashboard web funciona imediatamente após o `npm install`.

Preparar o ambiente

Antes de abrir o projeto, instale as ferramentas abaixo e confirme cada instalação pelo terminal (no Windows, use o PowerShell; no macOS/Linux, o terminal padrão). Se algum comando não for reconhecido, o respectivo instalador normalmente oferece a opção de adicionar a ferramenta ao PATH — marque essa opção durante a instalação para evitar problemas mais adiante.

FERRAMENTA	VERSÃO	COMO VERIFICAR	ONDE BAIXAR
Python	3.10 ou superior	<code>python --version</code>	python.org/downloads
Node.js (só dashboard)	20 LTS ou superior	<code>node -v</code>	nodejs.org
VSCode	versão atual	<code>code --version</code>	code.visualstudio.com
Git (opcional)	qualquer recente	<code>git --version</code>	git-scm.com

Recomenda-se também instalar estas extensões do VSCode (menu de extensões, atalho **Ctrl+Shift+X**): **Python** e **Pylance** (intellisense e depuração do pipeline), **ESLint** e **Tailwind CSS IntelliSense** (edição do dashboard). Todas são gratuitas e oficiais.

Organização das pastas: extraia os dois pacotes do projeto em uma mesma pasta de sua preferência (por exemplo, C:\Projetos\sad-encceja ou ~/projetos/sad-encceja). Você deverá ver duas pastas irmãs: **projeto_knn_encceja** (Python/Streamlit) e **dashboard_encceja_nextjs** (Next.js).

SEÇÃO 3

Interface 1 — App Streamlit (Python)

Esta é a interface principal do trabalho acadêmico: ela executa o modelo K-NN treinado pelo pipeline e gera o parecer gerencial automaticamente. Todo o processo acontece no terminal integrado do VSCode, que você abre com o atalho **Ctrl + `** (ou menu **Terminal → New Terminal**). Os comandos abaixo devem ser executados dentro da pasta **projeto_knn_encceja**, aberta no VSCode por **File → Open Folder**.

1 Criar o ambiente virtual

O ambiente virtual isola as bibliotecas do projeto das do sistema, evitando conflitos de versão. Execute uma única vez:

```
> python -m venv .venv
```

2 Ativar o ambiente virtual

O comando varia conforme o sistema operacional. Após ativar, o terminal passa a exibir (.venv) no início da linha:

WINDOWS — POWERSHELL

```
> .venv\Scripts\Activate.ps1
```

WINDOWS — PROMPT DE COMANDO (CMD)

```
> .venv\Scripts\activate.bat
```

LINUX / MACOS

```
$ source .venv/bin/activate
```

PowerShell bloqueou o script? Erro comum: "a execução de scripts foi desabilitada neste sistema". Resolva com Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser, confirme com S e repita o passo 2. Alternativa: use o terminal **cmd** do VSCode (seta ao lado do botão + no painel de terminal).

3 Instalar as dependências

Com o ambiente ativado, instale pandas, scikit-learn, streamlit e demais bibliotecas listadas no requirements.txt (leva de 1 a 3 minutos):

```
(.venv) > pip install -r requirements.txt
```

4 (Opcional) Retreinar o pipeline

Se o pacote veio com as pastas modelos/ e data/ já preenchidas (arquivo modelo_knn.joblib presente), **pule este passo**. Caso contrário, gere a base e treine o modelo — o segundo comando leva alguns minutos, pois busca o melhor k entre 28 configurações com validação cruzada 5-fold:

```
(.venv) > python src/etl_preparacao.py  
(.venv) > python src/treino_knn.py
```

5 Iniciar a interface Streamlit

Suba o servidor local do aplicativo:

```
(.venv) > streamlit run app/app_streamlit.py
```

6 Usar o sistema

O navegador abre automaticamente em **http://localhost:8501** (se não abrir, copie o endereço exibido no terminal). Na aba **Simulador de Matrícula**, preencha o perfil do candidato e clique em **Prever desempenho** para ver as notas previstas, os vizinhos consultados e o parecer com nível de risco. Para encerrar, volte ao terminal e pressione Ctrl + C.

Interface 2 — Dashboard Next.js

O dashboard web é a versão mais visual do SAD: cinco módulos com gráficos interativos, simulador com **botões de preset** para demonstração ao vivo (perfis de alto risco e favorável) e o motor K-NN executando no próprio navegador. Ele não exige Python — apenas Node.js. Abra a pasta **dashboard_encceja_nextjs** no VSCode e use o terminal integrado.

1 Instalar as dependências

Na primeira execução, o npm baixa Next.js, React, Recharts e as demais bibliotecas (de 1 a 2 minutos com internet):

```
> npm install
```

2 Iniciar o servidor de desenvolvimento

O modo de desenvolvimento compila sob demanda e recarrega a página a cada alteração no código:

```
> npm run dev
```

3 Explorar os cinco módulos

Acesse <http://localhost:3000>. Na aba **Simulador K-NN**, experimente os dois presets de demonstração — **alto risco** e **favorável** — para ver a comparação lado a lado das previsões, dos vizinhos e dos pareceres, recurso ideal para a gravação do vídeo de demonstração. Para produção otimizada, use `npm run build` seguido de `npm start`.

Rodando as duas ao mesmo tempo

As interfaces são independentes e ocupam portas diferentes (8501 e 3000), portanto podem rodar simultaneamente sem conflito — útil para comparar resultados ou demonstrar as duas versões na gravação do vídeo. No VSCode, abra um segundo terminal clicando no botão **+** no canto superior direito do painel de terminal: mantenha o primeiro com o ambiente virtual ativado servindo o Streamlit e use o segundo, sem ambiente virtual, para o dashboard Next.js. Cada terminal exibe o endereço local da sua interface, e ambos podem ser interrompidos de forma independente com `Ctrl + C`.

Solução de problemas frequentes

A tabela a seguir concentra os erros mais comuns na primeira execução, com a causa provável e o comando de correção. Na grande maioria dos casos o problema se resolve ativando o ambiente virtual correto ou conferindo a

MENSAGEM / SINTOMA	CAUSA PROVÁVEL	SOLUÇÃO
'python' não é reconhecido...	Python fora do PATH (Windows)	Reinstale marcando Add Python to PATH ou troque python por py
error: externally-managed-environment	pip do sistema usado sem ambiente virtual	Crie e ative o .venv (passos 1–2 da Seção 3) antes do pip install
running scripts is disabled on this system	Política de execução do PowerShell	Set-ExecutionPolicy RemoteSigned -Scope CurrentUser ou use o terminal cmd
streamlit: command not found	Ambiente virtual inativo	Ative o .venv e reinstale com pip install -r requirements.txt
FileNotFoundError: modelo_knn.joblib	Modelo ainda não treinado	Execute python src/etl_preparacao.py e python src/treino_knn.py
Unsupported engine / npm ERR!	Versão do Node.js antiga	Instale o Node 20 LTS ou superior e reabra o terminal
Port 3000/8501 is already in use	Outro processo ocupa a porta	Next.js: npm run dev -- -p 3001 · Streamlit: acrescente -server.port 8502
Página em branco após npm run dev	Compilação ainda em andamento	Aguarde a mensagem Ready no terminal e recarregue o navegador (F5)

SEÇÃO 7

Checklist de verificação final

Percorra esta lista após concluir a instalação para garantir que o ambiente está completo e pronto para a demonstração. Cada item leva menos de um minuto para ser conferido e corresponde a um comportamento esperado das interfaces já validadas durante o desenvolvimento — se algum item falhar, volte à Seção 6 e localize a mensagem de erro correspondente no terminal.

- ☐ O Streamlit responde em **http://localhost:8501** com três abas visíveis
- ☐ A previsão no Simulador retorna notas, vizinhos e um parecer com nível de risco
- ☐ O dashboard responde em **http://localhost:3000** com cinco abas e gráficos renderizados
- ☐ Os presets “alto risco” e “favorável” produzem pareceres vermelho e verde, respectivamente
- ☐ A aba Modelo & Métricas exibe MAE 16,7 e a curva de busca do k (melhor: 21)

Próximo nível: para usar os microdados oficiais do INEP em vez da base sintética, baixe o arquivo MICRODADOS_ENCCEJA_2024_REG_NAC.csv no portal do INEP, coloque-o em data/ e repita o passo 4 da Seção 3 — o pipeline detecta o arquivo oficial automaticamente.

Por fim, vale saber onde está cada peça do projeto para consultas futuras ou para a escrita do relatório acadêmico. A tabela abaixo resume os artefatos gerados pelo pipeline e os pontos de integração entre as duas interfaces — todos os caminhos são relativos à raiz de cada pacote e podem ser abertos diretamente no explorador de arquivos do VSCode.

ARQUIVO	O QUE CONTÉM
projeto_knn_encceja/README.md	Documentação acadêmica completa: metodologia, decisões de modelagem e resultados
modelos/modelo_knn.joblib	Modelo K-NN treinado (usado pelo app Streamlit)
export/dados_dashboard.json	Amostra da base + parâmetros do scaler consumidos pelo dashboard web
dashboard_encceja_nextjs/src/lib/knn.ts	Motor K-NN em TypeScript, espelho do pipeline Python
GUIA_VSCODE.md	Versão resumida deste guia, em markdown, dentro do pacote do projeto

Ambiente pronto. Decisões melhores.

Com as duas interfaces rodando, você tem em mãos um sistema completo de apoio à decisão: previsão de desempenho por K-NN, identificação automática de risco e recomendações gerenciais acionáveis para cursinhos preparatórios do ENCCEJA.

:8501

Streamlit — Simulador, Base Histórica, Modelo

:3000

Next.js — 5 módulos com presets de demonstração

SAD ENCCEJA · K-NN · MICRODADOS INEP 2024 · TRABALHO AV1 — SISTEMAS DE APOIO À TOMADA DE DECISÃO